

In a *self-attention language model*, each word from one language is attending to all the words in context of the other language, but the attention weights (or edges) represent to what degree each word is attending to (i.e., related to) each other word. Hence, a self-attention language model can reveal the meaning of a translated Chinese word with respect to the words in an English sentence. For example, the Chinese word 吃 means “eat,” so this Chinese word would have a large attention weight to “eat” but would only weakly be attending to the other words.

Self-attention makes more intuitive sense when used in language models, but in this book we’ve mostly dealt with machine learning models that operate on visual data, such as pixels from a video frame. Visual data, however, is not naturally structured as a collection of objects or nodes that we can directly pass into a relational module. We need a way of turning a bunch of pixels into a set of objects. One way to do it would be to simply call each individual pixel an object. To make things more computationally efficient, and to be able to process the image into more meaningful objects, we can first pass the raw image through a few convolutional layers that will return a tensor with dimensions (C,H,W) for channels, height, and width. In this way, we can define the objects in the convolved image as vectors across the channel dimension, i.e., each object is a vector of dimension C, and there will be $N = H * W$ number of objects (figure 10.12).

Figure 10.12. A convolutional layer returns a series of convolutional “filters” stored in a 3-tensor with shape channels (i.e., the number of filters) by height by width. We can turn this into a set of nodes by taking slices along the channel dimension where each node is then a channel-length vector, for a total of height times width number of nodes. We package these into a new matrix of dimension N × C, where N is the number of nodes and C is the channel dimension.